MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión C del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_castillo_ihsahn
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/castillo_ihsahn/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código
¿Qué es la abstracción al modelar un objeto? Da un ejemplo de un dato que conservarías y otro que omitirías según el contexto.

Tu respuesta

```
La complejidad del objeto al ser compilado.
```

Corrección. La respuesta habla de la complejidad al compilar, y eso no tiene que ver con la abstracción. Abstraer es quedarse con lo esencial para el problema; por ejemplo, para calcular el área de un círculo se conserva el radio y se omite el color.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Para qué sirve el constructor y en qué se distingue de un método común en su declaración?

Tu respuesta

```
Es más eficiente que otros procesos.
```

Corrección. Decir que es más eficiente no explica el constructor. El constructor inicializa la instancia al crearla, lleva el mismo nombre que la clase y no declara tipo de retorno, ni siquiera void.

Pregunta 0.3 · Preguntas abiertas

Pregunta

¿Qué impide final en una variable? Si guarda una referencia a un arreglo, ¿impide cambiar sus elementos?

Tu respuesta

```
No impide cambiar sus elementos, depende del arreglo si se impide el final.
```

Corrección. Es correcto que final no impide cambiar los elementos del arreglo. Faltó decir lo que sí impide: reasignar la variable, es decir, sustituir la referencia, después de inicializarla. Lo de que 'depende del arreglo' introduce confusión.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Por qué == no basta para comparar el contenido de dos String? Explica qué usarías para comparar sus textos.

Tu respuesta

```
Porque es como decir que son iguales, es mejor utilizar menor o mayor que.
```

Corrección. Los String no se comparan con menor o mayor que. == compara identidad (si las dos referencias señalan al mismo objeto); para comparar el texto se usa equals.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿En qué se distinguen while y do-while? Explica qué ocurre si la condición es falsa desde el inicio.

Tu respuesta

```
Do-while evita bucles infinitos, si la condición de por sí es falsa no hace falta usar
ninguno.
```

Corrección. Do-while no evita bucles infinitos. while evalúa la condición antes del cuerpo y puede dar cero vueltas; do-while la evalúa después y da al menos una vuelta aunque la condición sea falsa desde el inicio.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
int[][] tabla = {{9, 4, 7}, {2, 6, 8}};
int[] fila = tabla[0];
int total = 0;
for (int i = 0; i < fila.length; i++) {
    fila[i] = fila[i] / 2;
    total += tabla[0][i];
    System.out.println(i + ":" + total);
}
```

```
System.out.println((double) (total / 2));
System.out.println("P" + fila[0] + tabla[1][0]);
```

Tu respuesta

```
([4] [2] [3]), ([1] [3] [4])
```

Corrección. Calcula bien la fila modificada {4, 2, 3}, que es la parte correcta que se puede identificar, pero no escribe ninguna de las líneas de salida. La salida exacta era 0:4 / 1:6 / 2:9 / 4.0 / P42, porque fila es tabla[0] y el cast se aplica después de 9 / 2.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] datos = {3, 6};
int total = 0;
for (int i = 0; i <= datos.length; i++)
    total += datos[i];
System.out.println(total);
```

Tu respuesta

Clasificación

```
Compila pero truena
```

Causa o salida

```
Falta el [0] en  total += datos[i];
```

Corrección. La clasificación 'compila pero truena' es correcta. La causa no lo es: no falta ningún [0]. El ciclo usa <= y llega a datos[2], fuera del arreglo (ArrayIndexOutOfBoundsException).

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
Integer piezas = 6;
System.out.println(piezas + 2);
System.out.println("P" + piezas);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
8
```

> **Corrección.** Clasifica bien como 'funciona' y la primera línea, 8, es correcta. Faltó la segunda línea, P6, y explicar la regla: con + 2 el Integer se desempaqueta para sumar, y con texto se concatena.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Incisos independientes: a/b en main; c en una clase válida. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
static int medir(int valor) {
    if (valor > 0) return valor;
}
```

Tu respuesta

Clasificación

```
No compila
```

Causa o salida

```
Falta el endif.
```

> **Corrección.** La clasificación 'no compila' es correcta. Java no tiene endif: el error es que falta un return de int cuando valor es cero o negativo, y el compilador revisa todos los caminos.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada lámpara tiene etiqueta fija y nace apagada. Cuenta las lámparas creadas. Escribe encender() y una consulta del estado; cada lámpara cambia por separado. Usa el enum dado.

```
enum Estado { APAGADO, ENCENDIDO, PARPADEO }
```

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿Qué tipo tiene el atributo de estado y con qué valor nace?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

Sin respuesta.

Justificaciones

Sin respuesta.

> **Corrección.** Sin respuesta.

R4 · Código con repetición

Pregunta

Devuelve la posición de la menor temperatura usando for; si se repite, devuelve la primera. Los índices empiezan en 0. El arreglo no es null, tiene al menos un elemento y puede incluir negativos. No lo modifiques.

```
static int posicionMinima(int[] temperaturas)
```

Ejemplos (argumentos → retorno):

{8, 3, 5} → 1 · {7} → 0 · {-2, -8, -1} → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

```
static int posicionMinima(int[] temperaturas)
for int {8, 3, 5}; 1
```

> **Corrección.** La cabecera está bien, se menciona for y no imprime, no lee ni modifica el arreglo. No hay ciclo, comparación, posición candidata ni return. Lo esperado era iniciar posicion = 0, recorrer desde i = 1, actualizar si temperaturas[i] < temperaturas[posicion] y devolver posicion.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Un equipo decide su mantenimiento según los ciclos de uso. Un valor menor o igual que 0 es "INVALIDO". Para los positivos, aplica la primera regla que corresponda, en el orden de la tabla. Usa if / else-if / else. No imprimas ni leas del teclado.

```
static String mantenimiento(int ciclos)
```

Regla o valor recibido	Texto devuelto
Menor o igual que 0	"INVALIDO"
Divisible entre 24	"COMPLETO"
Si no, divisible entre 12	"REVISION"
Si no, divisible entre 4	"BASICO"

Cualquier otro	"NINGUNO"

Tu respuesta

```
static String mantenimiento(int ciclos)
if ciclos <= 0
return "INVALIDO"
```

Corrección. La cabecera es correcta, trata bien el caso ciclos <= 0 → "INVALIDO", empieza bien la selección con if y no imprime. Pero solo hay return en un camino. Faltó la cadena else-if para los múltiplos de 24, 12 y 4 (con % == 0, en ese orden) y el else con "NINGUNO"; también faltan los paréntesis y las llaves.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.